# A dive into the Adaboost algorithm and its applications

Zeid Ombotimbe

June 23, 2025

## 1 Introduction

AdaBoost, short for Adaptive Boosting, is a powerful machine learning algorithm primarily used for classification tasks. The core idea behind AdaBoost is to iteratively train a sequence of weak learners (which are any ways to classify data that perform slightly better than random guessing) and then combine them in a way that results in a classifier significantly stronger than any individual weak learner. This algorithm has many advantages:

- High Accuracy: AdaBoost often yields highly accurate classifiers, especially when weak learners are judiciously selected and effectively combined.
- Resistance to Overfitting: AdaBoost tends to generalize well and is less susceptible to overfitting compared to some other complex models.
- Ease of Computation: The AdaBoost algorithm is relatively simple and can be efficiently computed even on low-end machines.

Consequently, AdaBoost is a valuable tool for researchers. However, general descriptions of the algorithm often fail to provide a comprehensive understanding of its inner workings. In the following sections, we will delve deeper into the mechanics of AdaBoost and provide clear, illustrative examples to facilitate a better grasp of the algorithm.

## 2 Formal description

The insights presented in the next two section draw heavily from Robert Schapire's seminal work on the AdaBoost algorithm [1] and M. Christopher's comprehensive text [2]. The algorithmic derivation discussed in section 2.3 is sourced from R. Rojas' informative article [3]. We aimed to elucidate AdaBoost's foundational concepts and practical applications in a coherent manner.

### 2.1 AdaBoost's approach to the classification problem

For a given set of features, $\mathbf{x}_i = (x_{i1}, \ldots, x_{ik})$, and target values, $y_i \in \{-1, 1\}$ :

$$(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_n, y_n)$$

We aim to construct a strong classifier $H(\mathbf{x}_i) \to \hat{y}_i$. In context of AdaBoost the strong classifier takes the form of linear combination of weak learners:

$$H(\mathbf{x}_i) = \operatorname{sign}\left[\alpha_1 h_1(\mathbf{x}_i) + \alpha_2 h_2(\mathbf{x}_i) + \cdots + \alpha_M h_M(\mathbf{x}_i)\right] = \operatorname{sign}\left(C_M(\mathbf{x}_i)\right)$$

Where

- $\alpha_m \in (0, +\infty)$ are the weights assigned to each weak learner.
- $h_m(\mathbf{x}_i) \to \hat{y}_i$ represents a weak learner. That is any function that predicts the target value better than random guess.
- $M$ denotes the number of estimators (weak learners) or the iteration.

The number of iteration $M$ and the algorithm by which weak learners $h_m$ are estimated are arbitrary and chosen by a researcher. For an algorith, typically, a decision tree with one level of depth, known as decision stump, is used. A decision stump can be thought of as a simple rule of thumb, such as the following.

The number of iteration $M$ and the algorithm by which weak learners $h_m$ are estimated are arbitrary and chosen by a researcher. For an algorith, typically, a decision tree with one level of depth, known as *decision stump*, is used. A decision stump can be thought of as a simple rule of thumb, such as the following.

$$h(\text{height}) = \begin{cases} male, & \text{height} > 170, \\ female, & \text{height} \leqslant 170, \end{cases}$$

As a standalone classifier, this rule performs poorly. However, it provides some useful insight that allows us to classify observations better than random guessing. By combining it with other similarly weak but somewhat useful rules, we can create a classifier that is valuable for our purposes.

AdaBoost is essentially a procedure that allows us to identify a set of $M$ such weak rules and then combine them into one strong classifier. By leveraging the strengths of many weak learners, AdaBoost constructs a robust and accurate classifier, demonstrating the power of collaborative learning in improving classification performance.

## 2.2 The algorithm

1. Choose the number of iteration $M$ and an algorithm to be used for estimating weak learners $h_j$.
2. Initialize weights for the first iteration as follows:

$$w_1^{(1)} = w_2^{(1)} = \cdots = w_n^{(1)} = \frac{1}{n}$$

3. For each iteration $m = 1, \ldots, M$:
   (a) Estimate a weak learner $h_m(\mathbf{x}_i)$ that minimizes the weighted sum of misclassified points $\epsilon_m = \sum_{i=1}^n w_i \cdot \mathbb{I}\{h_m(\mathbf{x}_i) \neq y_i\}$.
   (b) Calculate the coefficient $\alpha_m = \frac{1}{2} \ln \frac{1-\epsilon_m}{\epsilon_m}$.
   (c) Update the weights $w_i^{(m+1)} = w_i^{(m)} e^{-y_i \alpha_m h_m(\mathbf{x}_i)}$ and renormilize them so that their sum equals to one: $\sum_{i=1}^n w_i^{m+1} = 1$
4. Combine the estimated weak learners and their coefficients into the strong classifier:

$$H(\mathbf{x}_i) = \text{sign}\left[\alpha_1 h_1(\mathbf{x}_i) + \alpha_2 h_2(\mathbf{x}_i) + \cdots + \alpha_M h_M(\mathbf{x}_i)\right]$$

## 2.3 Deriviation

At iteration $m$ we have a linear combination of $(m-1)$ weak learners [3]:

$$C_{m-1}(\mathbf{x}_i) = \alpha_1 h_1(\mathbf{x}_i) + \alpha_2 h_2(\mathbf{x}_i) + \cdots + \alpha_{m-1} h_{m-1}(\mathbf{x}_i)$$

To complete this iteration we seek a new weak learner $h_m(\mathbf{x}_i)$, a coefficient $\alpha_m$ and way to update weights for the subsequent iteration $w_i^{(m+1)}$. To derive formulas for these, we introduce the *exponential loss* function $E_m$, which takes the following form:

$$E_m = \sum_{i=1}^n e^{-y_i C_m(\mathbf{x}_i)} = \sum_{i=1}^n e^{-y_i C_{m-1}(\mathbf{x}_i)} e^{-y_i \alpha_m h_m(\mathbf{x}_i)}$$

One of the reasons why the loss function is defined in this manner is because it greatly simplifies the derivation process and leads to concise formulas for $\alpha_m$ and $w_i^{(m)}$. For further insights, we recommend referring to [2].

Denoting

$$w_i^{(m)} = e^{-y_i C_{m-1}(\mathbf{x}_i)}, \tag{1}$$

we rewrite $E_m$ as:

$$E_m = \sum_{i=1}^n w_i^{(m)} e^{-y_i \alpha_m h_m(\mathbf{x}_i)}$$

We split $E_m$ into the sums over correctly classified observations ($h_m(\mathbf{x}_i) = y_i \;\Rightarrow\; y_i h_m(\mathbf{x}_i) = 1$) and misclassified observations ($h_j(\mathbf{x}_i) \neq y_i \;\Rightarrow\; y_i h_m(\mathbf{x}_i) = -1$):

$$E_m = \overbrace{\sum_{i:\, h_m(\mathbf{x}_i)=y_i} w_i^{(m)} e^{-\alpha_m}}^{\text{correctly classified}} + \overbrace{\sum_{i:\, h_m(\mathbf{x}_i)\neq y_i} w_i^{(m)} e^{\alpha_m}}^{\text{misclassified}}$$

By adding and subtracting $\sum_{i:\, h_m(\mathbf{x}_i)\neq y_i} w_i^{(m)} e^{-\alpha_m}$, we get:

$$E_m = \sum_{i=1}^{n} w_i^{(m)} e^{-\alpha_m} + \left(e^{\alpha_m} - e^{-\alpha_m}\right) \sum_{i:\, h_m(\mathbf{x}_i)\neq y_i} w_i^{(m)}$$

On the right hand side, only the term $\sum_{h_m(\mathbf{x}_i)\neq y_i} w_i^{(m)}$ depends on $h_m$. That implies that to minimize total error, we must find a $h_m$ that minimizes the weighted sum of all misclassified observations: $\epsilon = \sum_{h_m(\mathbf{x}_i)\neq y_i} w_i^m$.

To update the weights we recall (1) and write:

$$w_i^{(m+1)} = e^{-y_i C_m(\mathbf{x}_i)} = e^{-y_i C_{m-1}(\mathbf{x}_i)} \cdot e^{-y_i \alpha_m h_m(\mathbf{x}_i)} = w_i^m \cdot e^{-y_i \alpha_m h_m(\mathbf{x}_i)}$$

To determine the coefficient $\alpha_m$, we adopt the traditional approach of differentiating $E_m$ by $\alpha_m$ and setting the derivative to zero:

$$\frac{dE_m}{d\alpha_m} = \frac{1}{d\alpha_m}\left[e^{-\alpha_m}\left(\sum_{i:\, h_m(\mathbf{x}_i)=y_i} w_i^{(m)}\right) + e^{\alpha_m}\left(\sum_{i:\, h_m(\mathbf{x}_i)\neq y_i} w_i^{(m)}\right)\right] = 0$$

This simplifies to:

$$e^{-\alpha_m}\left(\sum_{i:\, h_m(\mathbf{x}_i)=y_i} w_i^{(m)}\right) = e^{\alpha_m}\left(\sum_{i:\, h_m(\mathbf{x}_i)\neq y_i} w_i^{(m)}\right)$$

Taking the logarithm of both sides, we derive:

$$-\alpha_m + \ln\left(\sum_{i:\, h_m(\mathbf{x}_i)=y_i} w_i^{(m)}\right) = \alpha_m + \ln\left(\sum_{i:\, h_m(\mathbf{x}_i)\neq y_i} w_i^{(m)}\right)$$

$$2\alpha_m = \ln\left(\sum_{i:\, h_m(\mathbf{x}_i)=y_i} w_i^{(m)}\right) - \ln\left(\sum_{i:\, h_m(\mathbf{x}_i)\neq y_i} w_i^{(m)}\right)$$

$$\alpha_m = \frac{1}{2}\ln\left(\frac{\sum_{i:\, h_m(\mathbf{x}_i)=y_i} w_i^{(m)}}{\sum_{i:\, h_m(\mathbf{x}_i)\neq y_i} w_i^{(m)}}\right)$$

The last expression can be rewritten as:

$$\alpha_m = \frac{1}{2}\ln\left(\frac{1-\epsilon}{\epsilon}\right)$$

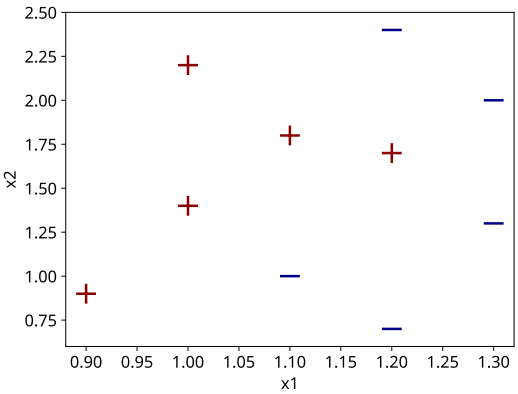
Where $\epsilon = \sum_{i:\, h_m(\mathbf{x}_i)\neq y_i} w_i^{(m)}$

# 3 A toy example

In this section, we will perform the AdaBoost algorithm step-by-step using a simple two-dimensional dataset presented in Table 1.

***table 1:*** *toy data*

| x1 | x2 | y |
|---|---|---|
| (0.9 | 0.9) | 1 |
| (1.0 | 1.4) | 1 |
| (1.0 | 2.2) | 1 |
| (1.1 | 1.8) | 1 |
| (1.2 | 1.7) | 1 |
| (1.1 | 1.0) | -1 |
| (1.2 | 0.7) | -1 |
| (1.2 | 2.4) | -1 |
| (1.3 | 1.3) | -1 |
| (1.3 | 2.0) | -1 |




***picture 1:*** *scatter plot of data*

## 3.1 First iteration

We begin by initializing the weights $w_i^{(1)} = \dfrac{1}{10}$:

***table 2:*** *toy data with initialized weights*

| x1 | x2 | w1 | y |
|---|---|---|---|
| (0.9 | 0.9) | 0.1 | 1 |
| (1.0 | 1.4) | 0.1 | 1 |
| (1.0 | 2.2) | 0.1 | 1 |
| (1.1 | 1.8) | 0.1 | 1 |
| (1.2 | 1.7) | 0.1 | 1 |
| (1.1 | 1.0) | 0.1 | -1 |
| (1.2 | 0.7) | 0.1 | -1 |
| (1.2 | 2.4) | 0.1 | -1 |
| (1.3 | 1.3) | 0.1 | -1 |
| (1.3 | 2.0) | 0.1 | -1 |

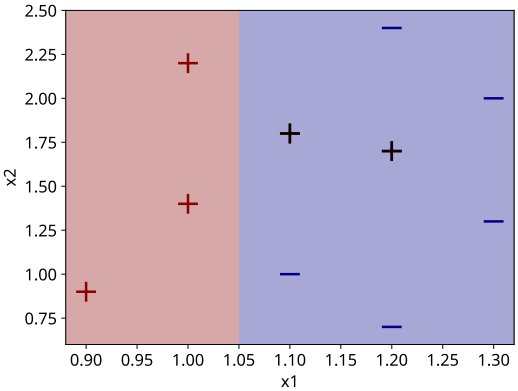
Next, we find the best weak learner ($h_1$), that minimizes the weighted sum of misclassified observations ($\epsilon = \sum_{i=1}^{n} w_i^1 \cdot I\{y \neq h_1(x_i)\}$). For this example, we will use a decision stump. Applying this to our data, the first weak learner is defined as:

$$h_1(\mathbf{x}_i) = \begin{cases} 1, & x_{i1} \leqslant 1.05 \\ -1, & x_{i1} > 1.05 \end{cases}$$

We will record the predicted values in the table 3 and provide a visual representation in picture 2

***table 3:*** *Toy data with values predicted by first weak learner*

| x1 | x2 | w1 | y | h1 |
|---|---|---|---|---|
| (0.9 | 0.9) | 0.1 | 1 | 1 |
| (1.0 | 1.4) | 0.1 | 1 | 1 |
| (1.0 | 2.2) | 0.1 | 1 | 1 |
| (1.1 | 1.8) | 0.1 | 1 | -1 |
| (1.2 | 1.7) | 0.1 | 1 | -1 |
| (1.1 | 1.0) | 0.1 | -1 | -1 |
| (1.2 | 0.7) | 0.1 | -1 | -1 |
| (1.2 | 2.4) | 0.1 | -1 | -1 |
| (1.3 | 1.3) | 0.1 | -1 | -1 |
| (1.3 | 2.0) | 0.1 | -1 | -1 |




***picture 2:*** *visual representation of the first weak learner*

We observe that our weak learner performs quite well, correctly classifying 8 out of 10 observations with only 2 false negatives. This a promising start. Furthermore, we appreciate its simplicity: a single rule that is straightforward to apply.

Now that we have the predicted values, we proceed to calculate the error, $\epsilon_1$:

$$\epsilon_1 = \sum_{i=1}^{n} w_i^{(1)} \cdot \mathbb{I}\{y = h_1(\mathbf{x}_i)\}$$

To facilitate this, we will add a column indicating whether each observation was misclassified, $\mathbb{I}\{y \neq h_1(\mathbf{x}_i)\}$ and the product $w_1 \cdot \mathbb{I}\{y \neq h_1(\mathbf{x}_i)\}$:

***table 4:*** *The calculation of the error for the first iteration*

| w1 | y | h1 | I{y != h1} | w1 * I{y = h1} |
|---|---|---|---|---|
| 0.1 | 1 | 1 | 0 | 0.0 |
| 0.1 | 1 | 1 | 0 | 0.0 |
| 0.1 | 1 | 1 | 0 | 0.0 |
| 0.1 | 1 | -1 | 1 | 0.1 |
| 0.1 | 1 | -1 | 1 | 0.1 |
| 0.1 | -1 | -1 | 0 | 0.0 |
| 0.1 | -1 | -1 | 0 | 0.0 |
| 0.1 | -1 | -1 | 0 | 0.0 |
| 0.1 | -1 | -1 | 0 | 0.0 |
| 0.1 | -1 | -1 | 0 | 0.0 |
| SUM | -- | -- | -- | 0.2 |

The error $\epsilon_1$ is equal to the sum of the last column (0.2). With this information, we can determine the coefficient $\alpha_1$:

$$\alpha_1 = \frac{1}{2} \ln\left(\frac{1 - 0.2}{0.2}\right) \approx 0.6931$$

Using $\alpha_1$, we update the weights for the next iteration:

$$w_i^{(2)} = w_i^{(1)} \cdot e^{-\alpha_1 y_i h_1(x_i)} = e^{-0.6931 \cdot y_i h_1(\mathbf{x}_i)}/10$$

Then, we normalize the weights:

$$w_i^{(2)} = w_i^{(2)} / \sum_{i=1}^{n} w_i^{(2)}$$

This process yields:

***table 5:*** *Updating weights for the second iteration*

| w1 | y | h1 | y * h1 | w2 |
|---|---|---|---|---|
| 0.10 | 1 | 1 | 1 | 0.0625 |
| 0.10 | 1 | 1 | 1 | 0.0625 |
| 0.10 | 1 | 1 | 1 | 0.0625 |
| 0.10 | 1 | -1 | -1 | 0.2500 |
| 0.10 | 1 | -1 | -1 | 0.2500 |
| 0.10 | -1 | -1 | 1 | 0.0625 |
| 0.10 | -1 | -1 | 1 | 0.0625 |
| 0.10 | -1 | -1 | 1 | 0.0625 |
| 0.10 | -1 | -1 | 1 | 0.0625 |
| 0.10 | -1 | -1 | 1 | 0.0625 |

Here we clearly observe how AdaBoost weighting operates. It prioritizes observations that have been misclassified. In the next iteration, the weak learner will be incentivized to generate a hypothesis that correctly classifies the false positives or false negatives from the previous iteration, even at the expense of a lower overall accuracy rate. Let's examine how these weights influence the next iteration.

## 3.2 Second iteration

***table 6:*** *toy data with the weights for the second iteration*

| x1 | x2 | w2 | y |
|---|---|---|---|
| 0.9 | 0.9 | 0.0625 | 1 |
| 1.0 | 1.4 | 0.0625 | 1 |
| 1.0 | 2.2 | 0.0625 | 1 |
| 1.1 | 1.8 | 0.2500 | 1 |
| 1.2 | 1.7 | 0.2500 | 1 |
| 1.1 | 1.0 | 0.0625 | -1 |
| 1.2 | 0.7 | 0.0625 | -1 |
| 1.2 | 2.4 | 0.0625 | -1 |
| 1.3 | 1.3 | 0.0625 | -1 |
| 1.3 | 2.0 | 0.0625 | -1 |

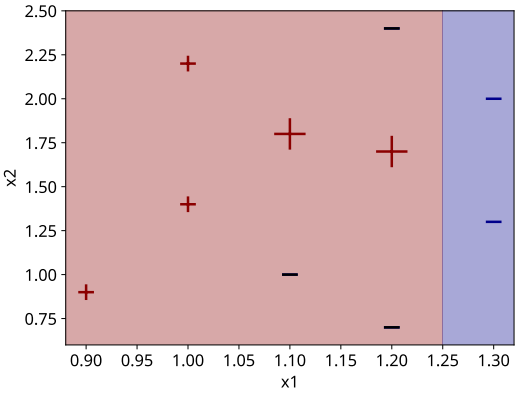
Running the decision stump algorithm yields the following rule:

$$h_2(x_i) = \begin{cases} 1, & x_{1i} \leqslant 1.25 \\ -1, & x_{1i} > 1.25 \end{cases}$$

We will add this rule to a column and plot it on a scatter plot:

***table 7:*** *toy data with target values predicted by the second weak learner*

| x1 | x2 | w2 | y | h2 |
|---|---|---|---|---|
| 1.0 | 1.4 | 0.0625 | 1 | 1 |
| 1.0 | 2.2 | 0.0625 | 1 | 1 |
| 1.1 | 1.8 | 0.2500 | 1 | 1 |
| 1.2 | 1.7 | 0.2500 | 1 | 1 |
| 1.1 | 1.0 | 0.0625 | -1 | 1 |
| 1.2 | 0.7 | 0.0625 | -1 | 1 |
| 1.2 | 2.4 | 0.0625 | -1 | 1 |
| 1.3 | 1.3 | 0.0625 | -1 | -1 |
| 1.3 | 2.0 | 0.0625 | -1 | -1 |




***picture 3:*** *visual representation of the second weak learner*

This time, we have 3 false positives, but we correctly classify the 4th and 5th observations. How will this affect the error and alpha? It turns out the error will decrease, and alpha will increase, making this classifier more significant.

***table 8:*** *The calculation of the error for the second iteration*

| w2 | y | h1 | I{y != h1} | w2 * I{y = h1} |
|---|---|---|---|---|
| 0.0625 | 1 | 1 | 0 | 0.0000 |
| 0.0625 | 1 | 1 | 0 | 0.0000 |
| 0.0625 | 1 | 1 | 0 | 0.0000 |
| 0.2500 | 1 | 1 | 0 | 0.0000 |
| 0.2500 | 1 | 1 | 0 | 0.0000 |
| 0.0625 | -1 | 1 | 1 | 0.0625 |
| 0.0625 | -1 | 1 | 1 | 0.0625 |
| 0.0625 | -1 | 1 | 1 | 0.0625 |
| 0.0625 | -1 | -1 | 0 | 0.0000 |
| 0.0625 | -1 | -1 | 0 | 0.0000 |
| SUM | -- | -- | -- | 0.1875 |

Despite this weak learner being less accurate, its error is smaller than the error of the previous one. Why is this so? Because in the first iteration, we established a basic classification that performed reasonably well but made two mistakes. Now, we are focusing on resolving those mistakes. To achieve this, we adjusted the weights in a way that emphasizes misclassified observations much more and correctly classified ones much less. The objective is for the new learner to find a rule that correctly predicts the target values for the 4th and 5th observations, even if this rule makes more mistakes overall compared to the previous one. As we observe, this is precisely what it accomplished.

Now that we have the error, we can calculate alpha:

$$\alpha_2 = \frac{1}{2} \ln \left( \frac{1 - 0.0625}{0.0625} \right) \approx 0.7332$$

Updating weights:

***table 9:*** *Updating weights for the third iteration*

| w1 | y | h1 | y * h1 | w3 |
|---|---|---|---|---|
| 0.0625 | 1 | 1 | 1 | 0.0385 |
| 0.0625 | 1 | 1 | 1 | 0.0385 |
| 0.0625 | 1 | 1 | 1 | 0.0385 |
| 0.2500 | 1 | 1 | 1 | 0.1538 |
| 0.2500 | 1 | 1 | 1 | 0.1538 |
| 0.0625 | -1 | 1 | -1 | 0.1667 |
| 0.0625 | -1 | 1 | -1 | 0.1667 |
| 0.0625 | -1 | 1 | -1 | 0.1667 |
| 0.0625 | -1 | -1 | 1 | 0.0385 |
| 0.0625 | -1 | -1 | 1 | 0.0385 |

The distribution of weights has shifted once again in favor of the misclassified observations. Now, the new learners will attempt to correct errors from both this current iteration and the previous one. Let's proceed quickly with another cycle and then move straight to the results.

## 3.3 Third, fourth and fifth iterations

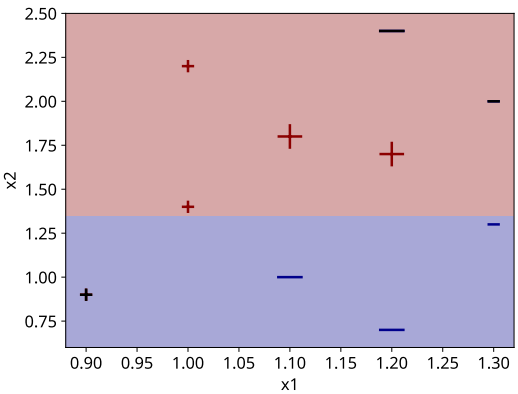
The weak learner for the third iteration is:

$$h_3(x_i) = \begin{cases} 1, & x_{2i} \leqslant 1.35 \\ -1, & x_{2i} > 1.35 \end{cases}$$

Putting it in a table and plotting:

***table 10:*** *toy data with target values predicted by the third weak learner*

| *x1* | *x2* | w2 | y | h3 |
|---|---|---|---|---|
| *0.9* | *0.9* | 0.04 | 1 | -1 |
| *1.0* | *1.4* | 0.04 | 1 | 1 |
| *1.0* | *2.2* | 0.04 | 1 | 1 |
| *1.1* | *1.8* | 0.15 | 1 | 1 |
| *1.2* | *1.7* | 0.15 | 1 | 1 |
| *1.1* | *1.0* | 0.17 | -1 | -1 |
| *1.2* | *0.7* | 0.17 | -1 | -1 |
| *1.2* | *2.4* | 0.17 | -1 | 1 |
| *1.3* | *1.3* | 0.04 | -1 | -1 |
| *1.3* | *2.0* | 0.04 | -1 | 1 |




***picture 4:*** *visual representation of the third weak learner*

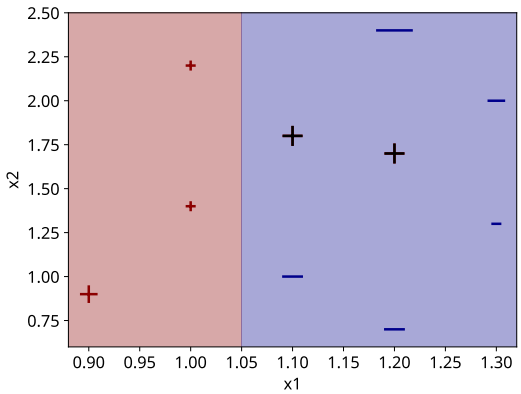
The error $\epsilon_3$ computes to 0.2436 and the coefficient $\alpha_3$ to 0.5665. Let's quickly review the results of last two iterations:

$$h_4(x_i) = \begin{cases} 1, & x_{1i} \leqslant 1.05 \\ -1, & x_{1i} > 1.05 \end{cases}$$

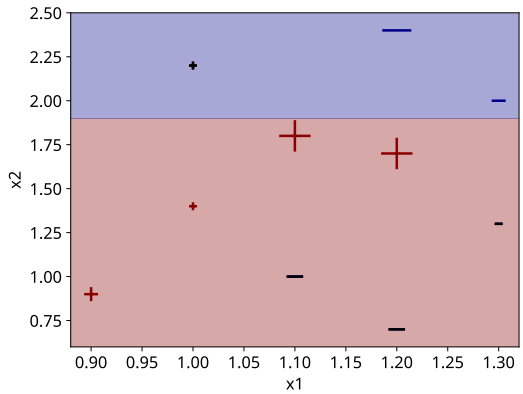
$$\epsilon_4 \approx 0.2033 \quad \alpha_4 \approx 0.6826$$

$$h_5(x_i) = \begin{cases} 1, & x_{2i} \leqslant 1.9 \\ -1, & x_{2i} > 1.9 \end{cases}$$

$$\epsilon_5 \approx 0.1702 \quad \alpha_5 \approx 0.7921$$




***picture 5:*** *visual representation of a fourth weak learner*




***picture 6:*** *visual representation of fifth a weak learner*

Composing all the weak learners and their corresponding coefficients into the linear combination $C(\mathbf{x}_i)$:

$$C(\mathbf{x}_i) = 0.6931 \cdot h_1(\mathbf{x}_i) + 0.7332 \cdot h_2(\mathbf{x}_i) + 0.5665 \cdot h_3(\mathbf{x}_i) + 0.6826 \cdot h_4(\mathbf{x}_i) + 0.7921 \cdot h_5(\mathbf{x}_i)$$

We construct a strong classifier by applying a sign function to it:

$$H(x_i) = \text{sign}\left[C(x_i)\right]$$

Finally, we put the results in the table

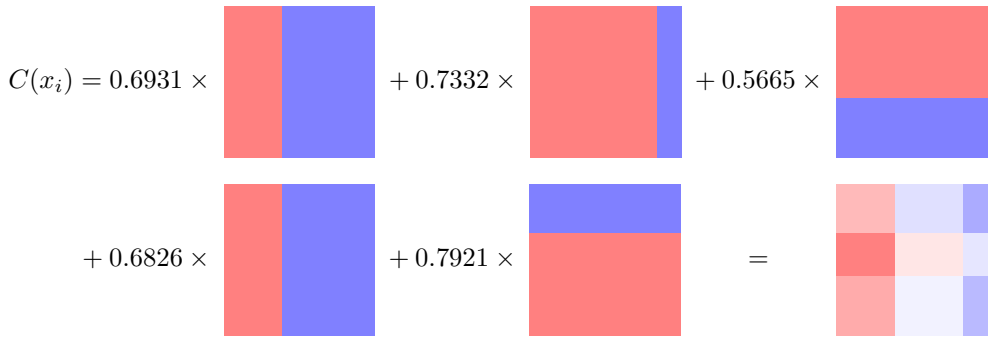
***table 11:*** *Linear combination $C(\mathbf{x}_i)$ and strong classifier $H(\mathbf{x}_i)$*

| C | H | y |
|---|---|---|
| 2.33 | 1 | 1 |
| 3.47 | 1 | 1 |
| 1.88 | 1 | 1 |
| 0.72 | 1 | 1 |
| 0.72 | 1 | 1 |
| -0.42 | -1 | -1 |
| -0.42 | -1 | -1 |
| -0.87 | -1 | -1 |
| -1.88 | -1 | -1 |
| -2.33 | -1 | -1 |

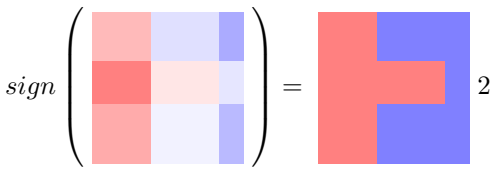
Great! We have successfully trained our model, and it has predicted all the classes perfectly. It took only 5 weak learners to achieve this result. While this might not be the most efficient result, as a decision tree can reach 100% accuracy in just 3 steps, the true strength of AdaBoost lies not in brevity, but in its ability to create highly accurate models while avoiding overfitting.
Now, let's come to the exiciting stuff: the visual representation of the algorithm

## 3.4 Visual representation of the solution

In this subsection, we will leverage the two-dimensionality of our example to provide a visualization of the AdaBoost algorithm. The reader has already seen how the decision stumps are depicted on the scatter plots and understands how AdaBoost combines them to build a strong classifier. Thus, it will be straightforward to grasp the visual representation of the AdaBoost algorithm:

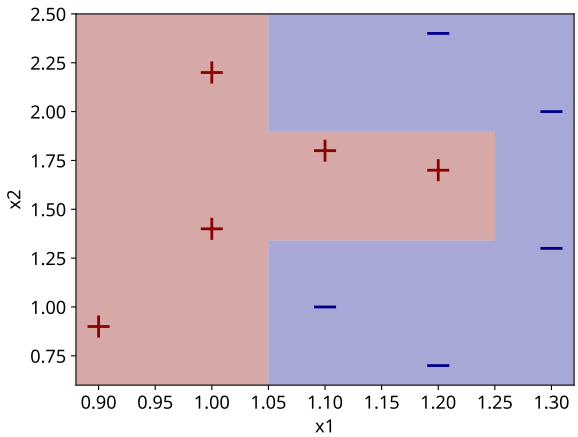



Taking sign function we will get:




Or putting it on a scatter plot:

***picture 7:*** *scatter plot of the solution*



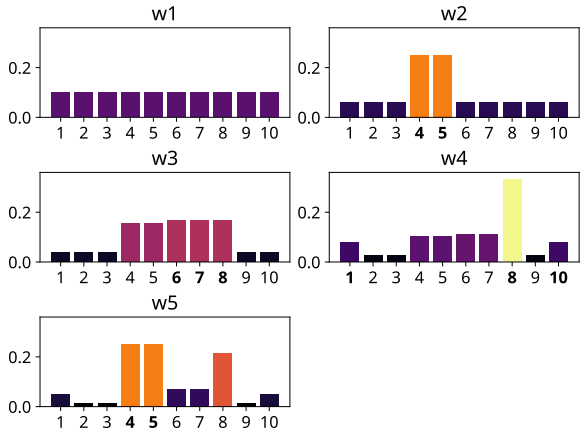

Amazingly, we observe how a set of simple rules, each incapable of predicting all target values accurately on its own, collectively forms a model that classifies all observations with high precision. This phenomenon is a remarkable demonstration of emergence: the combined model exhibits properties that none of its individual components possess independently.

### 3.4.1 Algorithm dynamics

A valuable approach to deepening our understanding of the AdaBoost algorithm's inner workings is to examine how its solutions and parameters evolve over time. We begin by analyzing the changes in the weights:

***table 12:*** *weights redistribution over iteration*

| w1 | w2 | w3 | w4 | w5 |
|---|---|---|---|---|
| 0.10 | 0.06 | 0.04 | 0.08 | 0.05 |
| 0.10 | 0.06 | 0.04 | 0.03 | 0.02 |
| 0.10 | 0.06 | 0.04 | 0.03 | 0.02 |
| 0.10 | 0.25 | 0.15 | 0.10 | 0.25 |
| 0.10 | 0.25 | 0.15 | 0.10 | 0.25 |
| 0.10 | 0.06 | 0.17 | 0.11 | 0.07 |
| 0.10 | 0.06 | 0.17 | 0.11 | 0.07 |
| 0.10 | 0.06 | 0.17 | 0.34 | 0.21 |
| 0.10 | 0.06 | 0.04 | 0.03 | 0.02 |
| 0.10 | 0.06 | 0.04 | 0.08 | 0.05 |

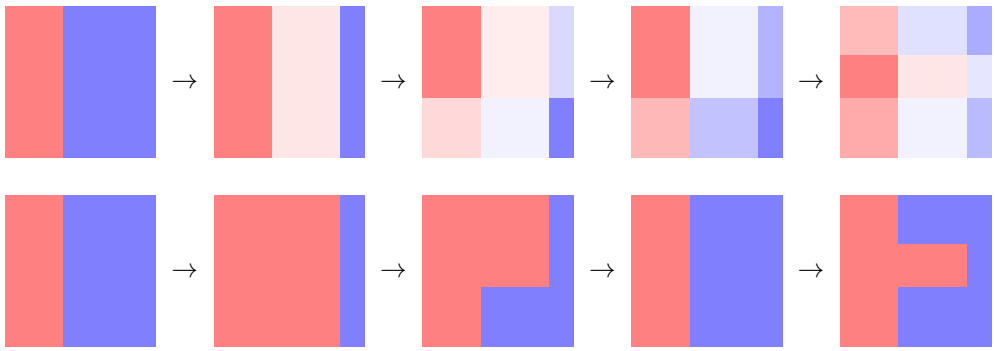



***picture 8:*** *visual representation of the weights redistribution. Numbers highlighted in bold indicate observations that were misclassified in the previous iteration.*

We start with equal weights, as the initial goal is to find any solution that works to some extent. Inevitably, this initial solution will make mistakes. Consequently, we adjust the weights to compel the new weak learner to devise a non-trivial solution that corrects these errors. This behavior is achieved through a straightforward rule: if an observation is correctly classified, we decrease its weight; if it is misclassified, we increase its weight.

This dynamic is clearly illustrated in Figure 8. On the x-axis, the numbers highlighted in bold represent the observations that were misclassified in the previous iteration. We observe how, in the second step, the weights of the 4th and 5th observations increase, while the weights of the others decrease. In the third step, the weights of the 6th, 7th, and 8th observations increase, with the weights of all other observations decreasing, and so on. This visualization effectively demonstrates the iterative weight adjustments that drive the AdaBoost algorithm towards a more accurate resultant classifier.

It also may be interesting to look at the visual dynamic of the solution which is depicted in the picture 9



***picture 9:*** *Visual dynamic of the solution. Upper row depicted the visual representation of the linear combination $C(\mathbf{x}_i)$ throught the iterations and the bottom the strong classifier $H = \operatorname{sign} C(\mathbf{x})$*

This diagram effectively illustrates how AdaBoost iteratively combines multiple simple hypotheses, gradually refining them to build a highly accurate classifier. It demonstrates the step-by-step process of weight adjustments and how the algorithm systematically corrects its mistakes, leading to an optimal classification model.

# 4 Real world example

## 4.1 Datasets overview

In this section, we explore the hyperparameters of the AdaBoost algorithm, their influence on estimation outcomes, and the computational complexities associated with them, utilizing two distinct datasets.
The first dataset, sourced from Kaggle [4], centers on car insurance and comprises 58,592 unique entries of individual policyholders. It encompasses 40 features, encompassing demographic specifics like age, vehicle attributes such as model and fuel type, and more (comprehensive descriptions available on Kaggle). The target variable, "claim status," indicates whether a claim was filed (1) or not (0).

***table 13:*** *A truncated view of a car insurance database*

| id | policy id | vehicle age | customer age | fuel type | model | ... | claim status |
|---|---|---|---|---|---|---|---|
| 1 | POL045360 | 1.2 | 41 | Diesel | M4 | ... | 0 |
| 2 | POL016745 | 1.8 | 35 | Diesel | M9 | ... | 0 |
| 3 | POL007194 | 0.2 | 44 | Diesel | M4 | ... | 0 |
| 4 | POL018146 | 0.4 | 44 | CNG | M1 | ... | 0 |
| 5 | POL049011 | 1.0 | 56 | Diesel | M5 | ... | 0 |
| ... | ... | ... | ... | ... | ... | ... | ... |
| 58588 | POL019269 | 2.6 | 48 | Petrol | M6 | ... | 0 |
| 58589 | POL001254 | 2.2 | 37 | Diesel | M4 | ... | 0 |
| 58590 | POL019859 | 2.2 | 35 | Petrol | M6 | ... | 0 |
| 58591 | POL014061 | 3.6 | 44 | Petrol | M6 | ... | 0 |
| 58592 | POL054184 | 0.4 | 49 | Petrol | M6 | ... | 0 |

The second dataset, also sourced from Kaggle [5], focuses on heart diseases. It includes data from respondents indicating whether they have a heart disease or have had a heart attack (1), alongside 21 features such as cholesterol levels, age, income, and others.

***table 14:*** *A truncated view of a heart disease dataset*

| id | HighBP | HighChol | Age | Education | Income | ... | HeartDiseasorAttack |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 9 | 4 | 3 | ... | 0 |
| 2 | 0 | 0 | 7 | 6 | 1 | ... | 0 |
| 3 | 1 | 1 | 9 | 4 | 8 | ... | 0 |
| 4 | 1 | 0 | 11 | 3 | 6 | ... | 0 |
| 5 | 1 | 1 | 11 | 5 | 4 | ... | 0 |
| ... | ... | ... | ... | ... | ... | ... | ... |
| 253675 | 1 | 1 | 5 | 6 | 7 | ... | 0 |
| 253676 | 1 | 1 | 11 | 2 | 4 | ... | 0 |
| 253677 | 0 | 0 | 2 | 5 | 2 | ... | 0 |
| 253678 | 1 | 1 | 7 | 5 | 1 | ... | 0 |
| 253679 | 1 | 1 | 9 | 6 | 2 | ... | 1 |

## 4.2 Tools used to perform AdaBoost algorithm

We will employ Python's `scikit-learn` package, which implements a variant of the AdaBoost algorithm designed to handle multiclass classification scenarios called "SAMME" [6]. It slightly differs from the one we used in the previous section. Notably, it introduces two new parameters in the calculation of alphas:

$$\alpha_j = \lambda \times \left( \ln \frac{1 - \epsilon_j}{\epsilon_j} + \ln(K - 1) \right)$$

Here $K$ is the number of classes (term $\ln(K-1)$ introduced by SAMME algorithm) and $\lambda$ is a learning rate (introduced by sklearn's implementation of algorithm [7]). A higher learning rate increases the contribution of each classifier.

Moreover, the weight updating mechanism is adjusted to accommodate multiclass scenarios:

$$w_i^{(m+1)} = w_i^{(m)} \cdot e^{\alpha_m \cdot \mathbb{I}\{y_i \neq h_m(i)\}}$$

These adjustments allow the algorithm to effectively handle multiple classes, though, may produce results different from a algorithm discussed earlier.

AdaBoost typically involves tuning three key hyperparameters:

- **Number of estimators (iterations):** This parameter controls the complexity of the model by determining how many weak learners (base estimators) are sequentially combined. More iterations can enhance model performance up to a certain point by reducing bias and capturing more intricate patterns. However, excessive iterations may lead to overfitting.
- **Classifier used to evaluate weak learner:** The choice of classifier affects the predictive capabilities of individual base estimators.
- **Learning rate:** This parameter scales the contribution of each weak learner in the final combination. Higher learning rates increase the impact of each estimator on the ensemble.

To assess the impact of these parameters, we will utilize three metrics:

- $Accuracy = \frac{TP+TN}{TP+FP+TN+FN}$. Shows what ratio of our prediction is correct.
- $Precision = \frac{TP}{TP+FP}$. Shows what proportion of the values model predicted as positive are actually positive. Measures the quality of positive predictions.
- $Recall = \frac{TP}{TP+FN}$. Show what proportion of all positive prediction our model classified as positive. It measures the ability to find all positive instances.

These metrics collectively provide insights into the model's predictive performance.

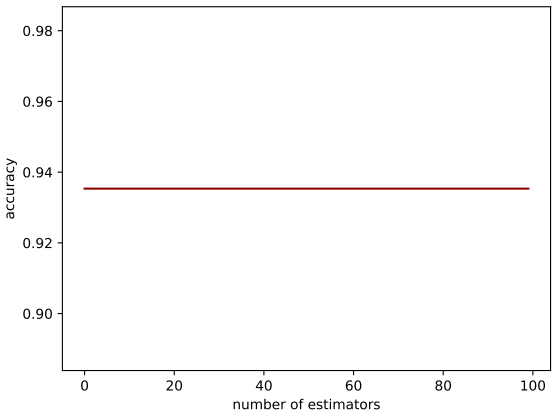
## 4.3 Problems introduced by Datasets

Before fitting our models, it's important to address two challenges posed by the datasets. Refer to figures 10a and 10b:

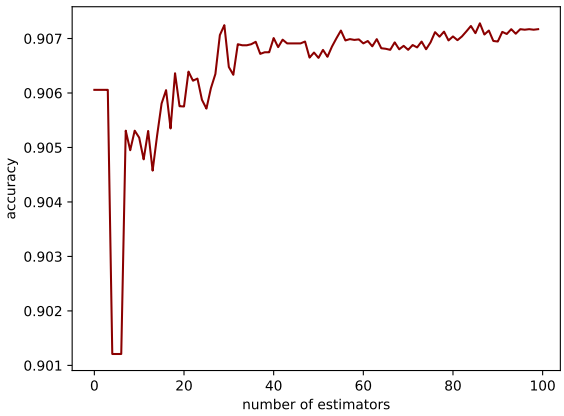
The accuracy of the insurance dataset remains unchanged regardless of the number of estimators, while the accuracy of the heart disease dataset shows insignificant differences. Such behavior may appear perplexing until we examine the imbalance between class 1 and class 0 instances in our datasets. For heart disease, the ratio is 23,893 to 253,680, and for insurance, it is 3,748 to 58,592. This imbalance makes it challenging to fit models, as AdaBoost may tend to classify all observations as 0 to optimize accuracy.

To rectify this issue, we utilize SMOTE (Synthetic Minority Over-sampling Technique), implemented in the `imblearn` Python package [8]. SMOTE generates synthetic observations to balance the dataset, thereby enhancing the reliability of our results. Figures 11a and 11b depict the accuracy estimates following oversampling, demonstrating significantly improved performance characteristics. This approach mitigates the challenges posed by class imbalance, ensuring a more robust and representative evaluation of model performance.

***picture 10:*** *accuracies of datasets before oversampling*

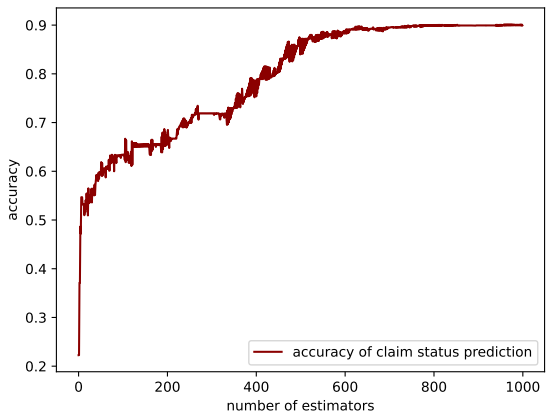



***(a)*** *accuracy of car insurance dataset depending on the number of estimators*

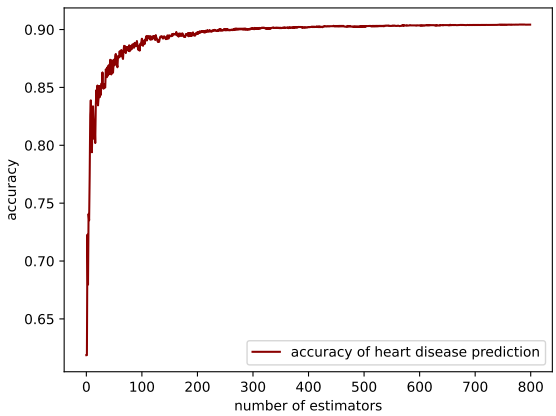



***(b)*** *accuracy of heart disease dataset depending on the number of estimators*

***picture 11:*** *accuracy of datasets after oversampling*

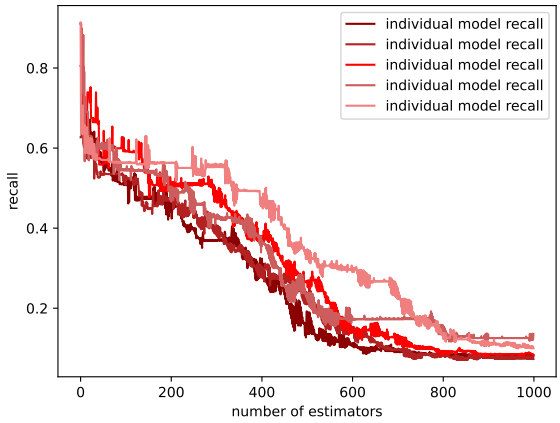



***(a)*** *accuracy of car insurance dataset after the oversampling*



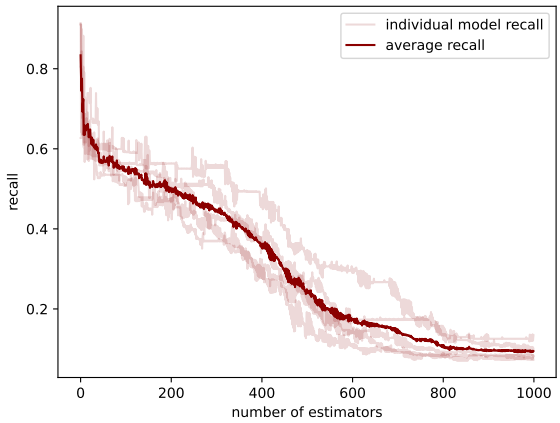

***(b)*** *accuracy of heart disease dataset after the oversampling*

Next, to evaluate our models, we partition the data into training and test samples. However, this introduces a new challenge: each random division yields different results. For instance, in Figure 12, we observe how the recall varies with the number of estimators for five models with identical parameters trained on five different random training samples of equal size. This variance underscores the impact of data partitioning on model performance.




***picture 12:*** *recall*

To mitigate this variability, we employ k-fold cross-validation. This technique involves dividing the dataset into k equally sized folds, using k-1 folds for training and the remaining fold for testing. We repeat this process k times, each time with a different fold as the test set. By averaging the metrics across these k iterations, we obtain a more robust evaluation of model performance, as depicted in Figure 13.




***picture 13:*** *test train with average*

With these considerations addressed, we can now proceed to explore how hyperparameters affect the model.

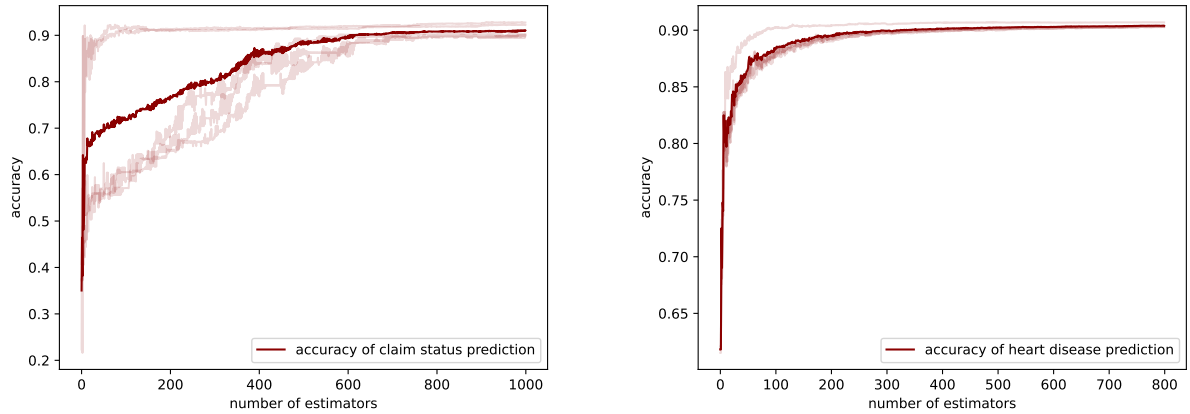
## 4.4 Hyperparameters overview

### 4.4.1 Number of estimators

Determining the optimal number of estimators is a crucial decision for researchers using AdaBoost, as it significantly impacts model performance, complexity, and computational time. Researchers often wonder how many iterations are sufficient for their dataset and whether additional estimators provide new valuable insights or reach a point of diminishing returns. They may speculate whether continuing to stack estimators could uncover insights that substantially enhance predictive capabilities. These are pertinent questions when employing AdaBoost.

To address these queries, we developed a library capable of parallel computation, enabling us to fit AdaBoost models with a up to 1000 number of estimators, even on extensive datasets. Let's delve into the results:

***picture 14:*** *accuracy of datasets depending on number of estimators*




***(a)*** *car insurance dataset* ***(b)*** *heart disease dataset*

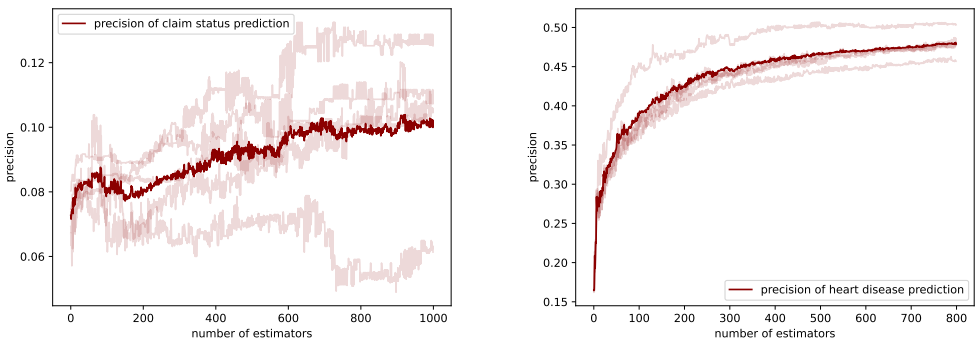
- **Car insurance:** we observed two scenarios: "lucky" divisions of training and test samples quickly reached a performance threshold around 60-80 estimators, while "unlucky" divisions steadily progressed to achieve a 0.9 accuracy score by approximately 700 estimators.
- **Heart disease:** conversely, for the heart disease dataset, all models reached their threshold around the 200th estimator, with one model demonstrating faster gains in predictive ability compared to others.

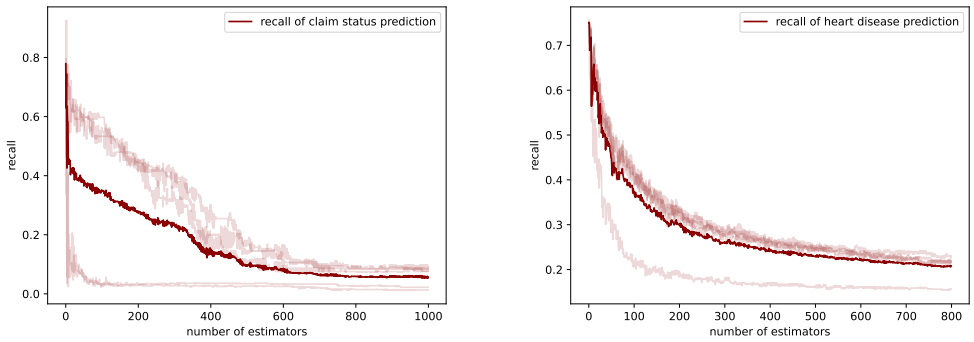
These observations underscore two key points: first, individual divisions of training and test samples can yield unreliable results, highlighting the importance of employing k-fold cross-validation. Second, both datasets exhibited a clear performance limit around a 0.9 accuracy rate that no model surpassed.

Based on these findings, we advise researchers to consider the computational resources available. If feasible, fitting models with a large number of estimators allows researchers to identify the point at which performance thresholds are met and subsequently truncate the model. For computationally intensive tasks on large datasets, it's prudent to assess the incremental gains in accuracy introduced by each additional estimator and cease model training once the gains become marginal (e.g., less than 0.005).

It's crucial to note that while accuracy is informative, especially on balanced datasets, it can be misleading on unbalanced datasets. Therefore, we also advocate evaluating precision and recall metrics to gain a more comprehensive understanding of model performance.

***picture 15:*** *precision and recall depending on the number of estimators*





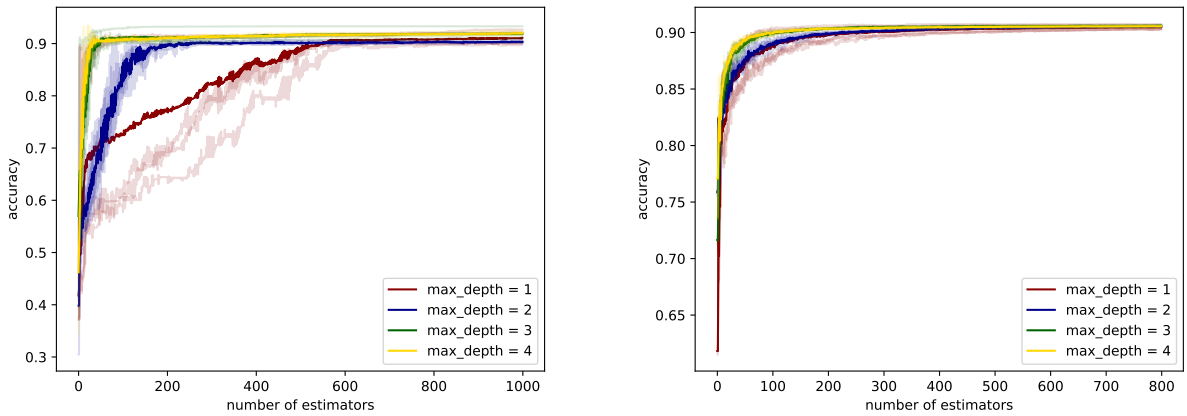

*(a) car insurance dataset* *(b) heart disease dataset*

- **Car insurance:** The precision of the car insurance dataset exhibits considerable variability and inconsistency across all folds, whereas recall shows more stability, with two distinct groups: two models starting with near-zero recall and three models gradually approaching 0.1 recall as the number of estimators increases. AdaBoost demonstrates limited predictive capability on this dataset, capturing only about 10% of all policies resulting in claims. Moreover, the quality of these predictions is notably low, with over 90% of predictions for positive claim status being false positives, rendering the model's predictions practically ineffective.
- **Heart disease:** the heart disease dataset yields more consistent and favorable outcomes. Across all folds, precision increases in a square root function manner with the number of estimators, plateauing around 600-700 estimators, indicating diminishing returns in precision gains. Recall, on the other hand, decreases hyperbolically with an asymptote around 0.2, albeit with one fold exhibiting outlier behavior. Overall, AdaBoost performs admirably on this dataset, producing a robust model capable of identifying up to 25% of all patients with heart disease, with 50% of positive predictions proving correct.

### 4.4.2 Decision tree maximal depth

While AdaBoost theoretically supports any algorithm for weak learner estimation, in practice, decision trees with a maximum depth of one—often referred to as decision stumps—are commonly employed. However, there are instances where experimenting with decision trees of greater depths may yield superior results.

***picture 17:*** *accuracy of datasets depending on the number of estimators for models with different decision tree maximal depth*

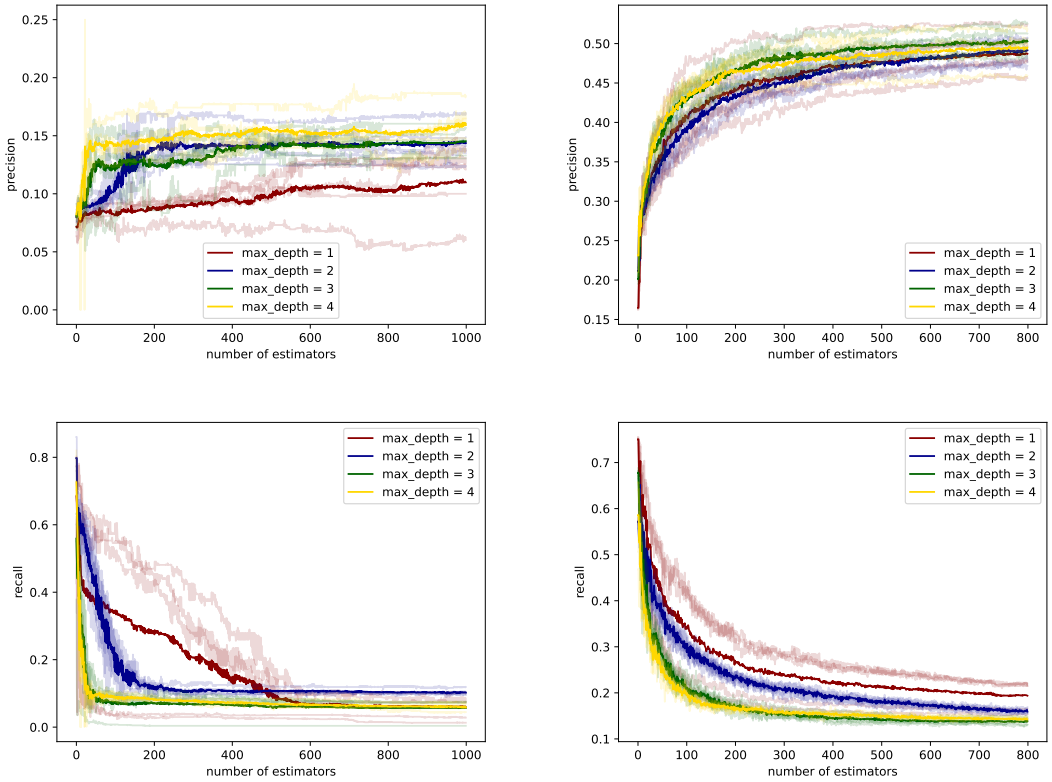



*(a) Car insurance dataset* *(b) Heart disease dataset*

- **Car insurance:** In the case of insurance data, increasing the maximum depth of the decision tree significantly enhanced model accuracy and expedited the attainment of performance thresholds compared to using decision stumps.

- **Heart diseases:** employing decision trees with higher depths yielded marginal benefits. Considering computational efficiency, using decision stumps might be more advantageous.

***picture 18:*** *precision and recall of datasets depending on the number of estimators for models with different decision tree maximal depth*




***(a)*** *car insurance dataset* ***(b)*** *heart disease dataset*

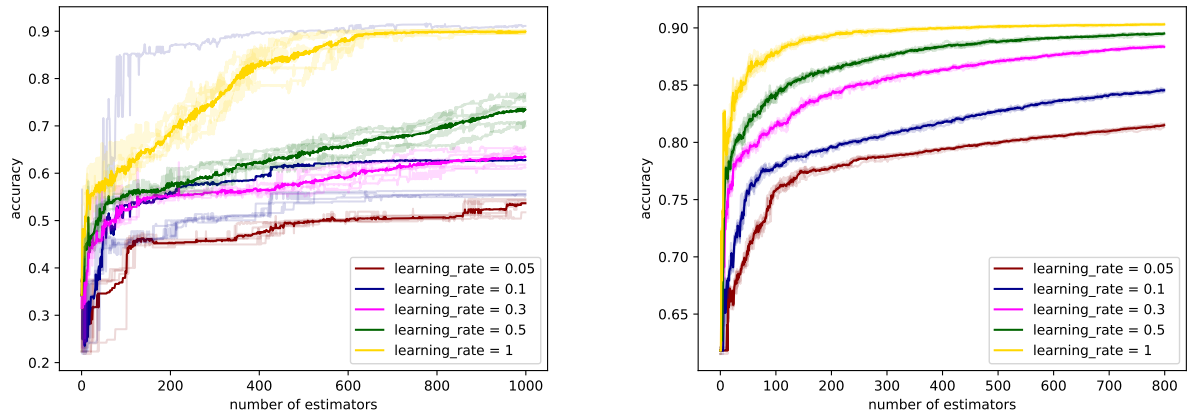
- **Car insurance:** Opting for a decision tree with higher depth increased the model's precision from 0.1 to 0.15. However, it led to a quicker decline in recall to approximately 0.1 compared to using decision stumps. Although this model remains suboptimal, it highlights how parameter tuning can potentially lead to acceptable solutions.

- **Heart desease:** From a precision perspective, a decision tree with depth 3 emerged as the optimal solution. Interestingly, models with a maximum depth of 1 outperformed those with a depth of 2, delivering the same precision with higher recall. This underscores the nuanced impact of decision tree depth on model performance and complexity, suggesting cautious consideration when increasing depth.

## 4.5 Learning rate

The learning rate in AdaBoost determines the influence of each weak learner on the overall classifier. Typically, a higher learning rate accelerates the convergence of the model.

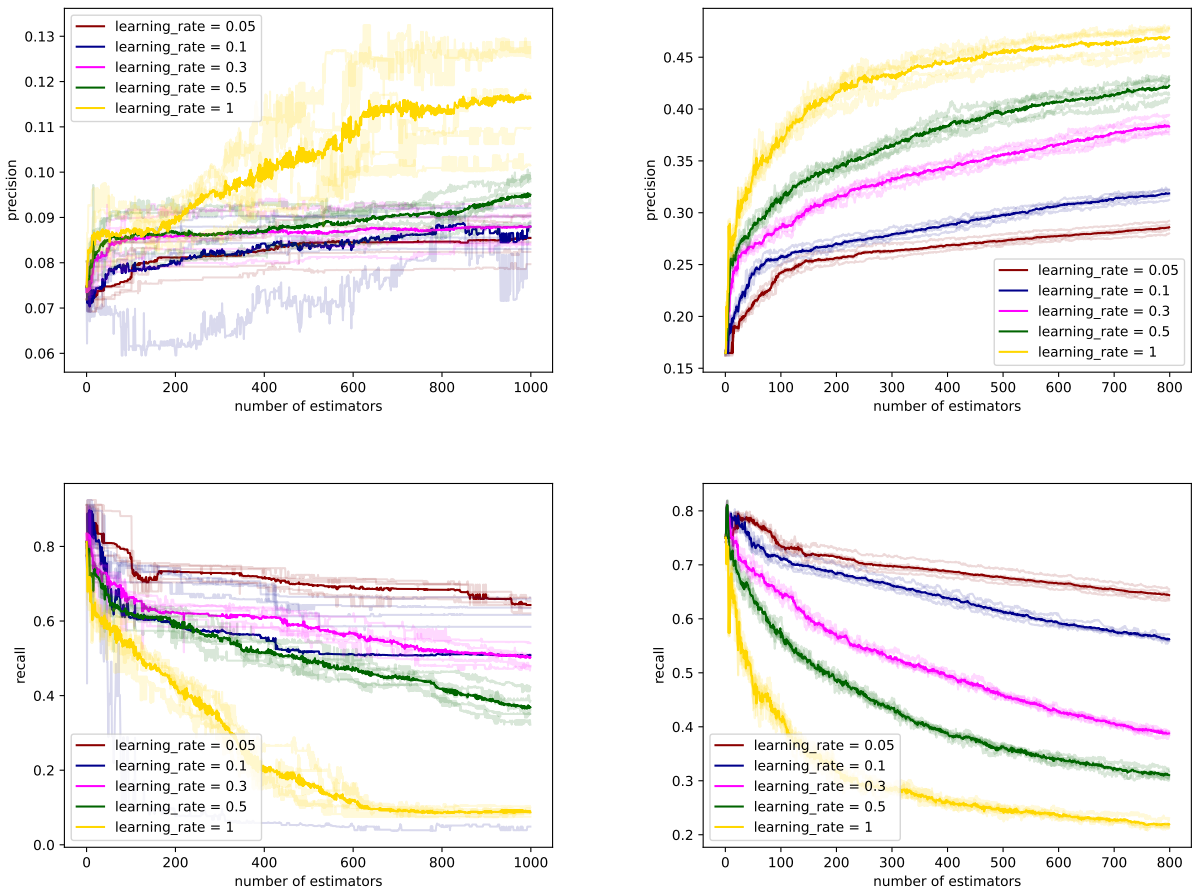
***picture 19:*** *accuracy of datasets depending on the number of estimators for models with different learn rate*




***(a)*** *Car insurance dataset* ***(b)*** *Heart diseases dataset*

Both datasets exhibit a similar pattern: a higher learning rate accelerates the rate of accuracy improvement.

***picture 20:*** *precision and recall of datasets depending on the number of estimators for models with different learn rate*



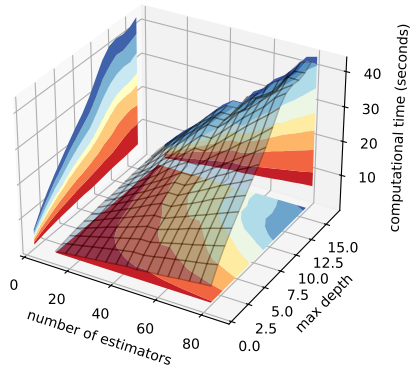

***(a)*** *car insurance dataset* ***(b)*** *heart disease dataset*

Furthermore, the trend holds across both datasets: increasing the learning rate enhances precision gain but at the expense of recall.

Therefore, researchers may consider slightly reducing the learning rate if maximizing true positives is a priority, even if it means sacrificing precision or overall quality of positive predictions.

## 4.6 Computational complexity and model tuning

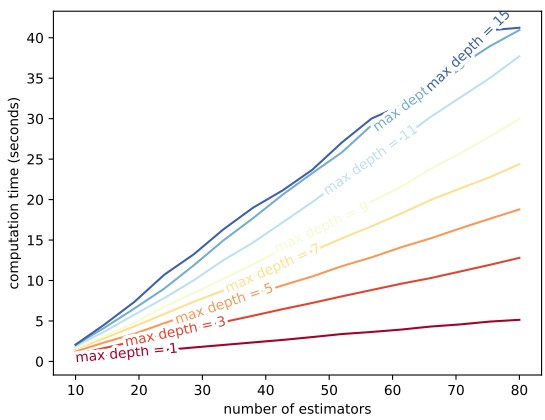
Figure 21 illustrates the relationship between the time required to fit the model on the heart disease dataset and the number of estimators and the maximal depth of the decision tree used.



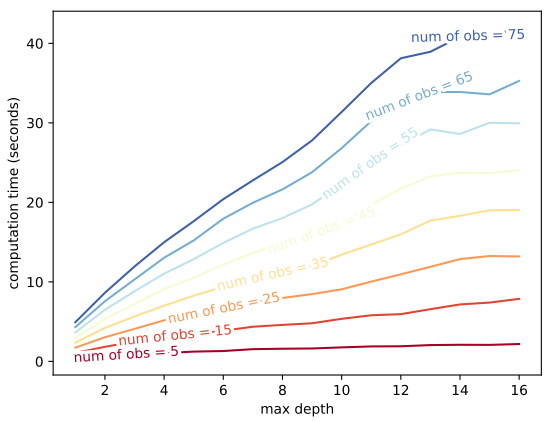

***picture 21:*** *Visual representation of the change of computation time depending on number of estimators and maximal depth*

The contour lines indicate that fixing one parameter while increasing the other results in a linear rise in computational time. Figures 22a and 22b provide a clearer visualization of this relationship. Notably, simultaneous increases in both parameters lead to a computational complexity that rises faster than linearly, as shown by the concave contour lines in Figure 22c. Therefore, when computational resources are constrained, optimizing the solution involves selecting either a high number of estimators with low maximal depth or vice versa.

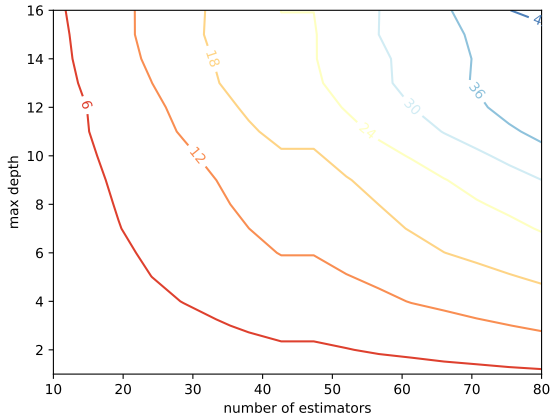
To determine the optimal combination for achieving the best results, whether optimizing for accuracy or precision, Figures 22d and 22e suggest that maximal depths between 2 and 5 paired with the highest possible number of estimators yield superior outcomes. Conversely, if higher recall is prioritized, Figure 22f indicates that opting for lower maximal depth is preferable, albeit at the cost of reduced accuracy and precision, as discussed in the previous subsection.



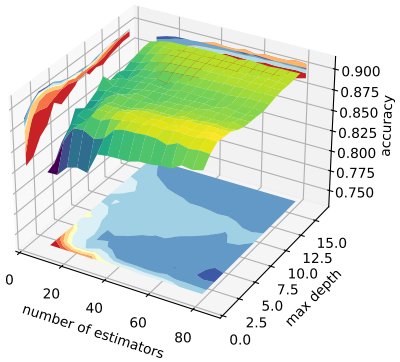

***(a)*** *Model computation time depending on the number of estimators for different decision tree maximal depths*



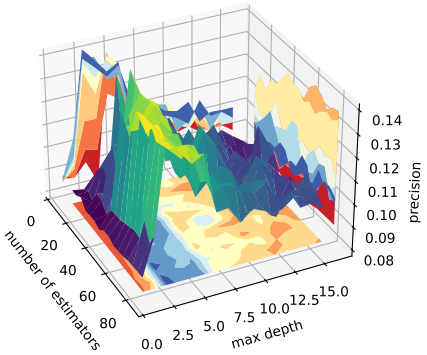

***(b)*** *Model computation time depending on the decision tree maximal depth for different number of estimators*



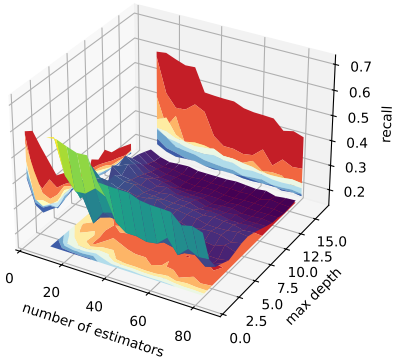

*(c) Contour lines of computational time for the maximal depth and number of estimators*




*(d) Accuracy dependence on number of estimators and maximal depth*




*(e) Precision dependence on number of estimators and maximal depth*




*(f) Recall dependence on number of estimators and maximal depth*

# 5 Conclusion

In conclusion, this article aims to provide readers with a comprehensive understanding of the AdaBoost algorithm and its practical implications. AdaBoost emerges as a potent tool in the researcher's toolkit, offering insights into effective ensemble learning strategies. By examining its application across car insurance claims and heart disease datasets, we uncovered valuable insights into parameter optimization and model performance evaluation.

For those seeking deeper insights, we recommend delving into the foundational work by the algorithm's creator, available in the book by Robert Schapire [1]. This resource provides further exploration into AdaBoost's theoretical underpinnings and practical applications, equipping researchers with tools to harness its potential across diverse domains.

# References


[1]R. E. Schapire **and** Y. Freund, *Boosting*, English, Adaptive Computation and Machine Learning series (MIT Press, London, England, 2012).

[2]M. Christopher, *Pattern recognition and machine learning*, English, 1 **edition**, Information Science and Statistics (Springer, New York, NY, 2006), **pages** 657–663.

[3]R. Rojas, "Adaboost and the super bowl of classifiers a tutorial introduction to adaptive boosting", English, (2009).

[4]S. Litvinenko, *Insurance claims dataset*, (2024) https://www.kaggle.com/datasets/litvinenko630/insurance-claims/data (**urlseen** 28/06/2024).

[5]A. Teboul, *Heart disease health indicators dataset*, (2022) https://www.kaggle.com/datasets/alexteboul/heart-disease-health-indicators-dataset (**urlseen** 28/06/2024).

[6]H. Zou, J Zhu, S Rosset **and** T Hastie, "Multi-class adaboost", English, Statistics and its Interface **2**, 349–360 (2009).

[7]python's sklearn package, *Multi-class adaboosted decision trees*, https://scikit-learn.org/stable/auto_examples/ensemble/plot_adaboost_multiclass.html (**urlseen** 28/06/2024).

[8]python's imblearn package, *Smote*, https://imbalanced-learn.org/dev/references/generated/imblearn.over_sampling.SMOTE.html (**urlseen** 28/06/2024).